



ASTRALIS UI

The Road Ahead

What is left, what comes next, and the recommended order — shipping, testing, the assistant, the theme builder, and the native-overlay opportunity.

8 July 2026

Companion: The Work Report.

Plain-language edition — concepts explained inline

1. Where we are, in one paragraph

The audit backlog is finished: everything broken is fixed, everything inconsistent is aligned, and the biggest component gaps are filled. What remains is a different kind of work — **shipping** (getting the library and docs into the world), **differentiating** (the features no competitor has yet), and **finishing touches** we consciously postponed. This document lists all of it, explains the ideas behind each item, and puts them in a recommended order.

2. The recommended order

#	Work	Size	Why this order
1	Foundations & Getting Started docs	days	The sidebar still says "soon" — embarrassing before anything ships
2	Deployment + publishing	days	Everything else (search, chatbot, real users) needs a live site and an installable package
3	Testing story	1–2 weeks	Confidence before growth; we have a gate for CSS but almost none for behavior
4	Astralis Assistant (the docs chatbot)	1–2 weeks	The agreed two-tier design; needs #2 first
5	Theme builder page	~1 week	Our best demo of the library's superpower
6	Native overlay migration	2–3 weeks	The "unclaimed opportunity" — worth doing right, not rushed
7	Next component wave	ongoing	DatePicker first (Calendar already exists for it)

3. Docs debt — the "soon" pages

Open the sidebar and look at the top: **Installation, Quick Start, Theming, Colors, Responsive Props, Style Props, Design Tokens** — all still marked `status: "soon"` in `navigation.ts`. We documented all 62 components but never wrote the guides that teach the *system*. These pages are cheap to write now because the two audit reports already contain the material — the Theming page is essentially the educational report's token-pyramid chapter, rewritten for users.

Also in this bucket: **keyboard tables for the older pages**. New pages (Menu, Combobox, NumberInput...) document their keyboard behavior in a table; the 50 pages written before that convention don't. One sweep brings them level.

***Concept — why keyboard tables matter commercially:** accessibility law in the EU (the European Accessibility Act, enforced since 2025) means companies now audit the component libraries they adopt. A library that documents its keyboard behavior per component reads as trustworthy; one that doesn't reads as a risk — even if the code is identical.*

4. Deployment and publishing

Nothing we built is reachable yet: the docs run on localhost and the package can't be installed.

The docs site needs a host (Vercel is the obvious choice for a Next.js site). Going live is also what unlocks: Algolia DocSearch (the free `Ctrl+K` search used by React/Tailwind/Vite docs — requires a public URL to crawl), the AI endpoints becoming useful (`/llms.txt` is only read by agents if it's on the internet), and the chatbot.

The package is currently blocked from publishing by one line — `"private": true` in `package.json`. Publishing also means choosing versioning discipline:

***Concept — semantic versioning (semver):** version numbers as a promise: `0.x` means "APIs may still change"; `1.0` means "breaking changes only with a major bump." Publishing as `0.1.0` is honest for where we are and keeps freedom to adjust APIs as real users arrive.*

A small decision to make here: the npm name. `astralis-ui` needs to be checked for availability (or scoped, like `@astralis/ui`).

5. Testing story

The CSS coverage gate proves every *style* exists, and every feature was verified in a browser when built — but there are almost no automated **behavior** tests. If someone refactors the overlay stack next month, nothing fails until a human notices.

The infrastructure is already installed (Vitest + Playwright browser mode, in `package.json`) — it's the tests that are missing. The highest-value targets are exactly the things we hand-verified this week, converted into scripts that run forever:

- Overlay stacking (Escape peels one layer; scroll unlocks only at the end).
- Menu keyboard flow (roving, disabled-skip, typeahead, focus return).
- NumberInput commit/clamp math and rapid-click stepping.
- Combobox filter/commit/revert.
- `generateBrandTokens` ramps and contrast decisions (pure function — trivial to test).

Concept — regression tests: a test isn't for proving the code works today (we did that manually); it's a tripwire so future changes can't silently un-fix it. The Po lesson — silent breakage is the expensive kind — applies to behavior exactly as it did to CSS.

6. The Astralis Assistant

The design we agreed on, ready to build once the docs are hosted:

- **Tier 0 — instant answers, free.** A curated bank of question→answer pairs (installation, theming, "how do I change the brand color"). Incoming questions are matched against it first; hits get a hand-written answer with exact code and links, instantly, at zero cost.
- **Tier 1 — the model.** Misses go to a small fast LLM that reads the relevant docs pages before answering. The retrieval source **already exists** — the same `docs-markdown.ts` that powers `/llms.txt`.
- **The loop that makes it compound:** every Tier-1 answer is cached by question. Frequent questions get reviewed, polished by hand, and promoted into Tier 0 — so the free tier grows from real traffic and the inference bill shrinks over time.

Needed from day one (cheap insurance): rate limits, a max-answer-length cap, and a scope guard ("I answer Astralis questions"). The chat UI itself is dogfooding — built from our own Drawer, CodeBlock, and (party trick) the runtime theming: "show me this in dark mode with a violet brand" can actually restyle the page.

***Concept — retrieval (RAG):** instead of hoping the model memorized your docs, you fetch the relevant pages at question time and hand them to the model as context. The model answers from your current text, so answers stay correct as the library changes.*

7. Theme builder page

A docs page where a visitor picks a brand color, radius, and dark/light — sees every component restyle **live** — then exports three things: the CSS variables, the `<AstralisProvider>` code, and a shareable preset URL an AI agent can consume ("apply theme astralis.dev/t/xK92f").

This is the flagship demo of our real differentiator: the whole theming system is runtime CSS variables, so the builder is mostly *plumbing we already have* — the hero's brand picker ([hero.tsx](#)) is a miniature of it, and `generateBrandTokens` in [provider.tsx](#) does the heavy lifting. Competitor proof this lands: shadcn's [/create](#) page and the tweakcn tool became hugely popular doing exactly this.

8. The native overlay migration (the big differentiator)

Our Modal, Popover, Tooltip, Menu and Select currently position and manage themselves with ~400 lines of our own JavaScript (the five hooks). The web platform now does most of that natively — and as of January 2026 it works in **every** browser engine:

*Concept — the native overlay platform, in four pieces: 1. The `popover` attribute — the browser puts the element in a special "top layer" above everything (no more z-index wars) and handles light-dismiss. 2. `<dialog>.showModal()` — a real modal: focus trap, backdrop, and Escape handling done by the browser. 3. **CSS anchor positioning** — "place this panel under that button, flip if there's no room" as pure CSS, replacing our positioning hook. 4. `@starting-style` — CSS enter/exit animations without the double-`requestAnimationFrame` trick our `usePresence` hook uses.*

The remarkable fact from our research: **no major library uses any of this yet** — their architectures predate it, and some (React Aria) are actively incompatible with it. We are unusually free to adopt it because our overlays sit on five swappable in-house hooks, not a third-party engine. The migration would *shrink* our code, delete the overlay-stack workarounds (the platform's top layer makes them unnecessary), and give Astralis a story no competitor tells: "CSS-native overlays."

Approach when we do it: one overlay at a time (Tooltip first — smallest), keep the current hooks as a fallback behind a feature check, and ship OddBird's small polyfill (~8 KB) for the last old browsers.

9. Next component wave

In order of leverage:

1. **DatePicker** — the highest-value quick(ish) win: it's an Input + Popover wrapped around the **Calendar we already built** (which even has full keyboard navigation now). Files to compose: `calendar/` + the anchor-positioning hook.
2. **AlertDialog** — a Modal preset for confirmations (`role="alertdialog"`, focus lands on Cancel). Small.
3. **Collapsible** — extract Accordion's grid-rows animation trick as a standalone single-panel component. Small.
4. **Toggle / ToggleGroup / SegmentedControl** — the Tabs `segmented` variant already proves the visual.
5. **HoverCard, FileUpload, Rating, EmptyState, CopyButton** — the next tier; CopyButton is fun because the docs site already hand-rolled one we can promote into the library.
6. **ContextMenu** — a right-click trigger variant of the existing Menu engine, not a new component.

10. Small leftovers (honest list)

Carried intentionally, none urgent:

- **MultiSelect's option row** is still its own copy (Select and Combobox share the new one) — unify when MultiSelect next changes.
- **Generic components still use** `forwardRef` (Box, Text, Button...) — fine on React 19; convert whenever each one is next touched.
- **The MCP server** (`packages/astralis-mcp/server.mjs`) reads docs from the local repo — after deployment it could optionally read the live site instead, so users who install the package (not the repo) can use it too.
- `KNOWN_ISSUES.md` is stale and should simply be deleted — the audit reports and this roadmap replace it.
- **Storybook stories** for the twelve new components (the docs demos cover human review; stories would feed visual regression testing later).

11. The one-line summary

Ship it (docs live, package published), harden it (tests), then differentiate it (assistant, theme builder, native overlays) — with DatePicker as the next component the moment Calendar's investment can pay off.